In [55]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from torch.utils.data import DataLoader, TensorDataset
```

In [15]:
```
#loading phishing data
filepath="C:\\Users\\lavan\\Desktop\\Machine Learning for Cybersecurity_ Lab 8-Phis
phishing_data = pd.read_csv(filepath)
```

In [19]:
```
#to display the whole data
#pd.set_option('display.max_rows', None)
#pd.set_option('display.max_columns', None)

# Then display the data
#print(phishing_data)
phishing_data.info(), phishing_data.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11055 entries, 0 to 11054
Data columns (total 31 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   having_IP_Address            11055 non-null  int64
 1   URL_Length                   11055 non-null  int64
 2   Shortining_Service           11055 non-null  int64
 3   having_At_Symbol             11055 non-null  int64
 4   double_slash_redirecting     11055 non-null  int64
 5   Prefix_Suffix                11055 non-null  int64
 6   having_Sub_Domain            11055 non-null  int64
 7   SSLfinal_State               11055 non-null  int64
 8   Domain_registeration_length  11055 non-null  int64
 9   Favicon                      11055 non-null  int64
 10  port                         11055 non-null  int64
 11  HTTPS_token                  11055 non-null  int64
 12  Request_URL                  11055 non-null  int64
 13  URL_of_Anchor                11055 non-null  int64
 14  Links_in_tags                11055 non-null  int64
 15  SFH                          11055 non-null  int64
 16  Submitting_to_email          11055 non-null  int64
 17  Abnormal_URL                 11055 non-null  int64
 18  Redirect                     11055 non-null  int64
 19  on_mouseover                 11055 non-null  int64
 20  RightClick                   11055 non-null  int64
 21  popUpWidnow                  11055 non-null  int64
 22  Iframe                       11055 non-null  int64
 23  age_of_domain                11055 non-null  int64
 24  DNSRecord                    11055 non-null  int64
 25  web_traffic                  11055 non-null  int64
 26  Page_Rank                    11055 non-null  int64
 27  Google_Index                 11055 non-null  int64
 28  Links_pointing_to_page       11055 non-null  int64
 29  Statistical_report           11055 non-null  int64
 30  Result                       11055 non-null  int64
dtypes: int64(31)
memory usage: 2.6 MB
```

Out[19]:
```
(None,
   having_IP_Address  URL_Length  Shortining_Service  having_At_Symbol  \
0                 -1           1                   1                 1
1                  1           1                   1                 1
2                  1           0                   1                 1
3                  1           0                   1                 1
4                  1           0                  -1                 1

   double_slash_redirecting  Prefix_Suffix  having_Sub_Domain  SSLfinal_State  \
0                        -1             -1                 -1              -1
1                         1             -1                  0               1
2                         1             -1                 -1              -1
3                         1             -1                 -1              -1
4                         1             -1                  1               1

   Domain_registeration_length  Favicon  port  HTTPS_token  Request_URL  \
0                           -1        1     1           -1            1
1                           -1        1     1           -1            1
2                           -1        1     1           -1            1
3                            1        1     1           -1           -1
4                           -1        1     1            1            1

   URL_of_Anchor  Links_in_tags  SFH  Submitting_to_email  Abnormal_URL  \
0             -1              1   -1                   -1            -1
1              0             -1   -1                    1             1
2              0             -1   -1                   -1            -1
3              0              0   -1                    1             1
4              0              0   -1                    1             1

   Redirect  on_mouseover  RightClick  popUpWidnow  Iframe  age_of_domain  \
0         0             1           1            1       1             -1
1         0             1           1            1       1             -1
2         0             1           1            1       1              1
3         0             1           1            1       1             -1
4         0            -1           1           -1       1             -1

   DNSRecord  web_traffic  Page_Rank  Google_Index  Links_pointing_to_page  \
0         -1           -1         -1             1                       1
1         -1            0         -1             1                       1
2         -1            1         -1             1                       0
3         -1            1         -1             1                      -1
4         -1            0         -1             1                       1

   Statistical_report  Result
0                  -1      -1
1                   1      -1
2                  -1      -1
3                   1      -1
4                   1       1  )
```

In [21]:
```
# Splitting data into features (X) and target (y)
X = phishing_data.drop(columns=['Result']).values
y = phishing_data['Result'].values
```

In [23]:
```
# Split the data into training (80%) and test (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [25]:
```
# Convert data to PyTorch tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)
```

In [27]:
```
# Creating DataLoader objects for batching
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)
```

In [29]:
```
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

In [31]:
```
# Define a simple neural network model
class PhishingModel(nn.Module):
    def __init__(self):
        super(PhishingModel, self).__init__()
        self.fc1 = nn.Linear(30, 64)  # 30 input features
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.sigmoid(self.fc3(x))
        return x
```

In [34]:
```
# Instantiate model, define loss function and optimizer
model = PhishingModel()
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

In [38]:
```
print(labels)
```

```
tensor([[ 1.],
        [-1.],
        [ 1.],
        [-1.],
        [ 1.],
        [ 1.],
        [ 1.],
        [ 1.],
        [ 1.],
        [ 1.],
        [-1.],
        [-1.],
        [-1.],
        [-1.],
        [-1.],
        [ 1.],
        [-1.],
        [-1.],
        [-1.],
        [-1.],
        [-1.],
        [-1.],
        [-1.],
        [ 1.],
        [-1.],
        [ 1.],
        [-1.],
        [ 1.],
        [ 1.],
        [ 1.],
        [ 1.],
        [-1.],
        [-1.],
        [-1.],
        [ 1.],
        [ 1.],
        [-1.],
        [-1.],
        [ 1.],
        [-1.],
        [-1.],
        [ 1.],
        [ 1.],
        [-1.],
        [-1.],
        [ 1.],
        [ 1.],
        [-1.],
        [ 1.],
        [-1.],
        [-1.],
        [ 1.],
        [ 1.],
        [ 1.],
        [ 1.],
        [-1.],
```

```
       [-1.],
       [-1.],
       [-1.],
       [ 1.],
       [ 1.],
       [ 1.],
       [-1.],
       [ 1.]])
```

In [49]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

Out[49]:
```
PhishingModel(
  (fc1): Linear(in_features=30, out_features=64, bias=True)
  (fc2): Linear(in_features=64, out_features=32, bias=True)
  (fc3): Linear(in_features=32, out_features=1, bias=True)
  (relu): ReLU()
  (sigmoid): Sigmoid()
)
```

In [51]:
```
# Training the model
epochs = 20
for epoch in range(epochs):
    model.train()
    running_loss = 0.0

    # Loop through batches in the training data
    for inputs, labels in train_loader:
        # Move inputs and labels to the same device as the model
        inputs, labels = inputs.to(device), labels.to(device)

        # Convert labels from -1/1 to 0/1
        labels = (labels == 1).float()

        # Zero the gradients
        optimizer.zero_grad()

        # Forward pass: compute predicted outputs by passing inputs to the model
        outputs = torch.sigmoid(model(inputs))

        # Compute the loss
        loss = criterion(outputs, labels)

        # Backward pass: compute the gradient of the loss with respect to model par
        loss.backward()

        # Perform a single optimization step (parameter update)
        optimizer.step()

        # Accumulate the training loss
        running_loss += loss.item()

    # Print the average loss for the current epoch
    print(f"Epoch {epoch+1}/{epochs}, Loss: {running_loss/len(train_loader):.4f}")
```

```
Epoch 1/20, Loss: 0.5865
Epoch 2/20, Loss: 0.5218
Epoch 3/20, Loss: 0.5180
Epoch 4/20, Loss: 0.5168
Epoch 5/20, Loss: 0.5150
Epoch 6/20, Loss: 0.5146
Epoch 7/20, Loss: 0.5130
Epoch 8/20, Loss: 0.5126
Epoch 9/20, Loss: 0.5118
Epoch 10/20, Loss: 0.5101
Epoch 11/20, Loss: 0.5098
Epoch 12/20, Loss: 0.5090
Epoch 13/20, Loss: 0.5096
Epoch 14/20, Loss: 0.5088
Epoch 15/20, Loss: 0.5085
Epoch 16/20, Loss: 0.5079
Epoch 17/20, Loss: 0.5070
Epoch 18/20, Loss: 0.5064
Epoch 19/20, Loss: 0.5065
Epoch 20/20, Loss: 0.5064
```

In [65]:

```python
# Set the model to evaluation mode
model.eval()

# Initialize lists to store predictions and true labels
y_true = []
y_pred = []

# Disable gradient calculation for evaluation
with torch.no_grad():
    # Loop through the test data
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        # Get model predictions
        outputs = model(inputs)
        predicted = (outputs >= 0.5).float()  # Convert probabilities to binary pre

        # Append true labels and predictions
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(predicted.cpu().numpy())

# Convert lists to numpy arrays for evaluation
y_true = np.array(y_true)
y_pred = np.array(y_pred)

# Calculate evaluation metrics with handling for undefined metrics
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='weighted', zero_division=0)  #
recall = recall_score(y_true, y_pred, average='weighted', zero_division=0)        #
f1 = f1_score(y_true, y_pred, average='weighted', zero_division=0)                #
conf_matrix = confusion_matrix(y_true, y_pred)

# Print the results
print(f'Accuracy: {accuracy:.4f}')
print(f'Precision: {precision:.4f}')
```

```
print(f'Recall (Sensitivity): {recall:.4f}')
print(f'F1-Score: {f1:.4f}')
print('Confusion Matrix:')
print(conf_matrix)
```

```
Accuracy: 0.5427
Precision: 0.5393
Recall (Sensitivity): 0.5427
F1-Score: 0.5410
Confusion Matrix:
[[   0  893   63]
 [   0    0    0]
 [   0   55 1200]]
```